GiftBox Studio

A configurable gift-box application

Software Design Patterns / Final Project Report

Team member	Group
Saparali Shalkar	SE-2532
Nurbayev Murager	SE-2532
Mukhametov Temirlan	SE-2532

Repository: https://github.com/Temirlan222/FinalSDP

Submission date: 11 November 2026. The repository is private; the lecturer must be granted access.

Introduction

GiftBox Studio lets a customer assemble a small, personal gift box, decorate individual gifts and choose a pricing offer. The application then creates an order and shows status notifications to two different audiences. Its purpose is to demonstrate six coordinated design patterns in a complete Java application with an accessible browser interface.

A gift box is a useful domain because several kinds of change occur independently: the catalogue can grow, gifts can receive optional presentation, discount policies can change, and different observers can react to order progress. Separating these responsibilities makes the relationships between the patterns visible without requiring a large business system.

What this submission contains

The project includes Java source, HTML/CSS/JavaScript assets, build scripts, automated tests, this report, editable UML sources and actual screenshots. All pricing and order rules run in Java. The browser sends selections and displays the server response; it does not replace the Java implementation with a separate pricing simulation.

Scope: a local educational demonstration. There is no payment processing, login, real delivery or outbound email. Orders remain in memory until the Java process stops.

Problem, scope and architecture

Functional requirements

The user selects one to twelve gifts from Book (1500 KZT), Candle (800 KZT) and Tea collection (1200 KZT). Each item can receive gift wrap for 200 KZT and a ribbon for 100 KZT. A recipient is required; a short message is optional. The application computes the decorated subtotal and applies exactly one offer to the whole box.

An order starts in CREATED, moves to PACKING and ends in READY. Each valid change notifies the customer view and the packing desk. Skipping stages, repeating a completed transition, creating an empty box and selecting unknown products are rejected.

Layer	Responsibility	Main files
Browser	Selection, accessible controls, responsive layout, display of Java results	web/index.html, style.css, app.js
HTTP adapter	Bounded form input, routing, JSON responses, local static assets	web/WebServer.java, Json.java
Application	Compose creation, decoration, building, pricing and order creation	facade/GiftBoxFacade.java
Domain	Products, wrappers, policies, box and order events	api, factory, builder, decorator, strategy, pricing, observer

Dependencies and runtime

The Java core needs JDK 11 or newer and no external libraries. The HTTP adapter uses the JDK HTTP server and listens on 127.0.0.1. The project can therefore be demonstrated offline after cloning or extracting it. JavaScript uses standard browser APIs, and all illustrations are local CSS shapes, so no remote assets are required.

Data and validation

Amounts are nonnegative whole KZT stored as int. Percentage calculations use a long intermediate and floor the final total. Math.addExact rejects overflow while adding gift and decoration prices. The builder trims text, limits the recipient to 60 characters and the message to 200, and copies its item list when building the result.

The box list is immutable, but this is not a deep copy of arbitrary Gift objects. Built-in products and decorators are immutable; third-party Gift implementations must keep their values stable. These assumptions are documented in CONTRACT.md.

Creating gifts and building a box

Factory Method

GiftCreator is the abstract creator. Its public createGift operation calls the protected createProduct factory method and checks the returned product. BookCreator, CandleCreator and TeaCreator override createProduct to supply different Gift implementations. The product classes expose only description and price and carry no decoration or discount logic.

```
public final Gift createGift() {
    Gift gift = createProduct();
    if (gift == null || gift.getPrice() < 0 || gift.getDescription().isBlank()) {
        throw new IllegalStateException("The creator returned an invalid gift");
    }
    return gift;
}
protected abstract Gift createProduct();
```

Evidence: factory/GiftCreator.java and the three concrete creator classes. The map in GiftBoxFacade chooses a creator by a UI key; that map is not the Factory Method. The pattern is the overridable creation operation used by the superclass. A test injects a new creator without modifying the facade algorithm.

Builder

GiftBox.Builder collects the recipient, message and gifts step by step. build validates the required state and constructs a GiftBox. The product takes a copy of the builder list, so later builder reuse cannot add items to an earlier box. Pricing remains outside the builder because it is a separate responsibility.

```
GiftBox.Builder builder = new GiftBox.Builder()
        .forRecipient(recipient).withMessage(message);
for (GiftSelection selection : selections) {
    builder.addGift(createDecoratedGift(selection));
}
GiftBox box = builder.build();
```

The excerpt illustrates the sequence in GiftBoxFacade.quote; its production loop also checks selection for null. The actual builder is in builder/GiftBox.java. Tests verify empty input, text limits, the 12-gift limit, list immutability and builder reuse.

Why both patterns are useful

Factory Method controls which individual product is created. Builder controls how an aggregate is assembled. They solve different problems and both appear in one normal request: the facade asks a creator for a Gift, decorates it, and adds it to a builder. A simple constructor would be enough for a fixed two-field object, but this box has optional text, repeated items and validation at completion.

Temirlan’s part: decoration and pricing

Decorator

Gift is the component interface. GiftDecorator also implements Gift and stores one wrapped Gift. GiftWrap and Ribbon are concrete decorators. Both delegate to the enclosed object, then extend its description and price. They can wrap products supplied by any creator without knowing their concrete class.

```
Gift decorated = new Ribbon(new GiftWrap(book));
// GiftWrap.getPrice():
return Math.addExact(super.getPrice(), 200);
// Ribbon.getPrice():
return Math.addExact(super.getPrice(), 100);
```

A book remains 1500 KZT; the wrapped object reports 1800 KZT. This avoids a subclass for every combination such as BookWithRibbonAndWrap. Reversing these two wrappers changes description order but not the sum. Repeated wrapping is possible in the domain API, while the UI deliberately offers each option once per item.

Strategy

DiscountStrategy declares applyDiscount(subtotal). PriceCalculator is the context: it stores a strategy, supports replacement and calls the interface after calculating the decorated subtotal. NoDiscount, StudentDiscount, HolidayDiscount and ThresholdDiscount are the concrete strategies.

Policy	Rule	Example
NoDiscount	Return subtotal	2600 → 2600
StudentDiscount	floor(subtotal × 90 / 100)	2600 → 2340
HolidayDiscount	floor(subtotal × 80 / 100)	2600 → 2080
ThresholdDiscount	Subtract 500 when subtotal ≥ 5000	4999 → 4999; 5000 → 4500

```
public int applyDiscount(int subtotal) {
    if (subtotal < 0) {
        throw new IllegalArgumentException("Subtotal cannot be negative");
    }
    return subtotal >= MINIMUM_SUBTOTAL ? subtotal - SAVING : subtotal;
}
```

A percentage parameter alone would handle the first percentage policies. The threshold policy deliberately uses a different algorithm, making interchangeable behavior more meaningful. The calculator is unchanged when the policy changes. The HTTP adapter still registers the available choices; open/closed design does not mean every file is forever unchanged.

Orders, events and the facade

Observer

Order is the publisher and maintains a list of OrderObserver subscribers. CustomerNotifications and PackingDesk implement the callback independently. Each records its own in-app history. subscribe avoids duplicate registrations; unsubscribe removes a subscriber. The publisher iterates a snapshot of the subscriber list while dispatching an event.

```
public void advanceTo(OrderStatus next) {
    Objects.requireNonNull(next, "status");
    if (next.ordinal() != status.ordinal() + 1) {
        throw new IllegalArgumentException("Follow CREATED -> PACKING -> READY");
    }
    status = next;
    for (OrderObserver observer : List.copyOf(observers)) {
        observer.onStatusChanged(id, status);
    }
}
```

The enum order defines the current linear workflow. Adding branching states would require an explicit transition policy. Events are synchronous and in memory. They are not network pushes or real email; the browser receives both updated histories after its status request.

Facade

GiftBoxFacade provides quote and placeOrder. The client supplies recipient text, gift selections and a discount strategy. The facade resolves creators, applies wrappers, feeds a builder and invokes PriceCalculator. placeOrder additionally constructs OrderReceipt, which owns an Order and attaches both observers. A quote has no order or notification side effects.

```
Quote quote = quote(recipient, message, selections, strategy);
return new OrderReceipt("GB-" + UUID.randomUUID(), quote);
```

Evidence: facade/GiftBoxFacade.java, OrderReceipt.java and observer/Order.java. The facade hides the assembly sequence from the browser and HTTP routing code. It does not take over every subsystem responsibility: builder validation, arithmetic and observer dispatch stay in their own classes.

One coordinated scenario

A single order exercises Factory Method, Decorator, Builder, Strategy and Facade. Later status updates exercise Observer on that same order. All six are operational within one application, although not every pattern executes at the same instant. This matches the assignment’s coordination requirement.

Principles and Clean Code decisions

Principle	Concrete evidence and consequence
Single Responsibility	GiftWrap changes presentation; PriceCalculator calculates money; GiftBox.Builder assembles a box; WebServer translates HTTP. Changing an HTTP response does not require editing a discount algorithm.
Open/Closed	A new DiscountStrategy works with PriceCalculator unchanged. A new GiftCreator can be injected into GiftBoxFacade. UI options and default registration still need updating.
Liskov Substitution	Products and decorators provide the same Gift operations and nonnegative prices. PartTests uses an independent Gift implementation to check that decorators and pricing do not depend on a particular factory.
Interface Segregation	Gift has two operations, DiscountStrategy one, OrderObserver one. Pricing does not depend on order status methods or browser controls.
Dependency Inversion	PriceCalculator depends on DiscountStrategy and Gift. Order depends on OrderObserver. The facade accepts a creator map; its convenience constructor wires concrete defaults at the composition boundary.
DRY and readable names	DiscountMath centralizes percentage validation and safe multiplication. createDecoratedGift, calculateSubtotal and advanceTo communicate intent. Shared form parsing and JSON output are isolated in the web layer.

Defensive behavior

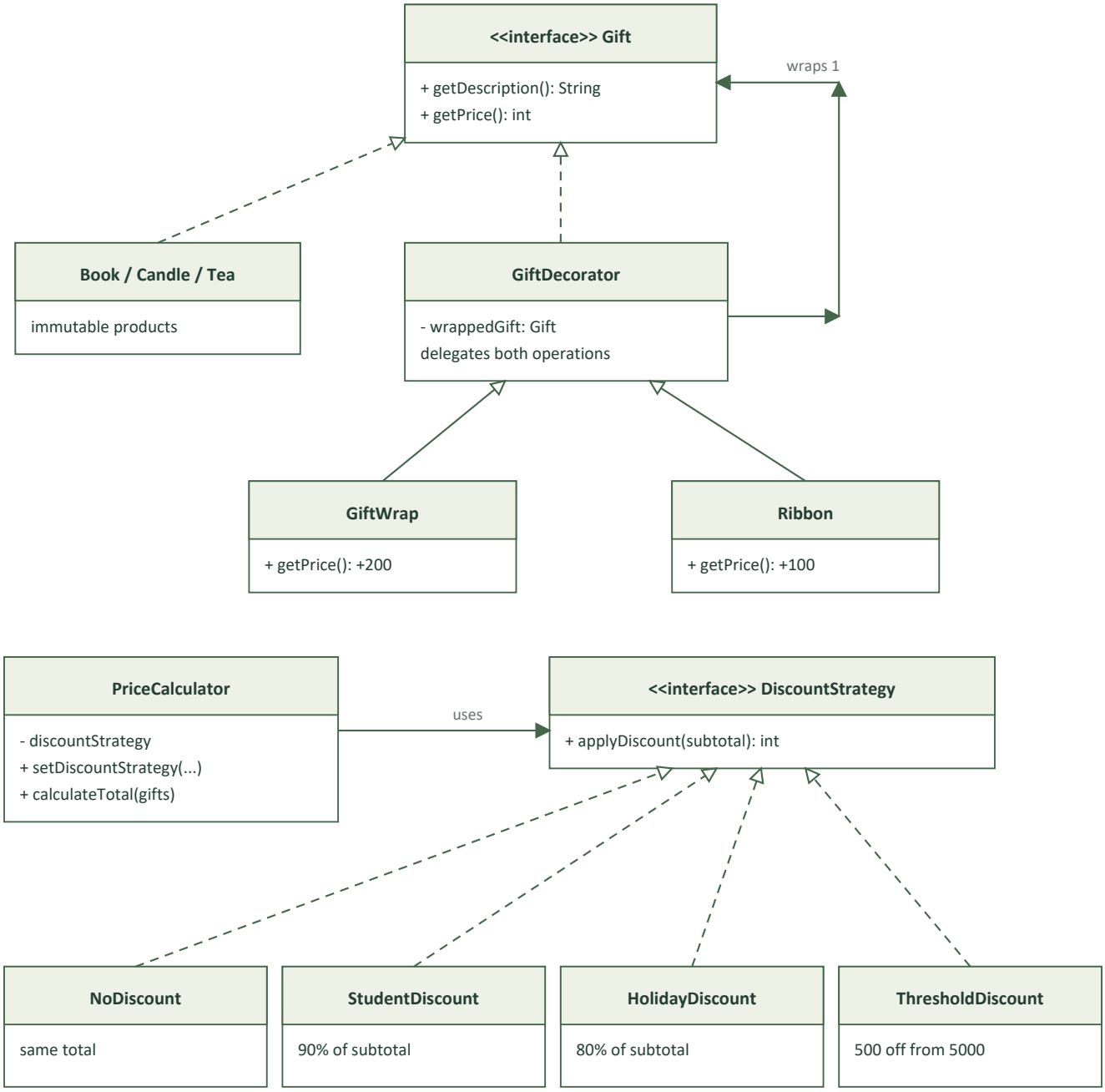
Invalid domain input is rejected near its owner. The HTTP layer limits request bodies, rejects unsupported methods and origins, and serves only an explicit list of local assets. The UI uses textContent for user text instead of injecting HTML, clears stale quote values while recalculating, and ignores older responses after a selection change.

Trade-offs, not perfection

The tiny output-only JSON encoder keeps the project dependency-free; it is not a general JSON parser. The HTTP server is intentionally sequential and local. Observer exceptions are not isolated: built-in callbacks are simple list appends, but a future unreliable subscriber needs error handling. These choices keep the defense manageable while making the extension points visible.

UML: Decorator and Strategy

Simplified class diagrams. A hollow triangle points to the parent or interface; a dashed line denotes interface implementation. Plain arrows show associations or use. Methods not needed for the pattern are omitted.

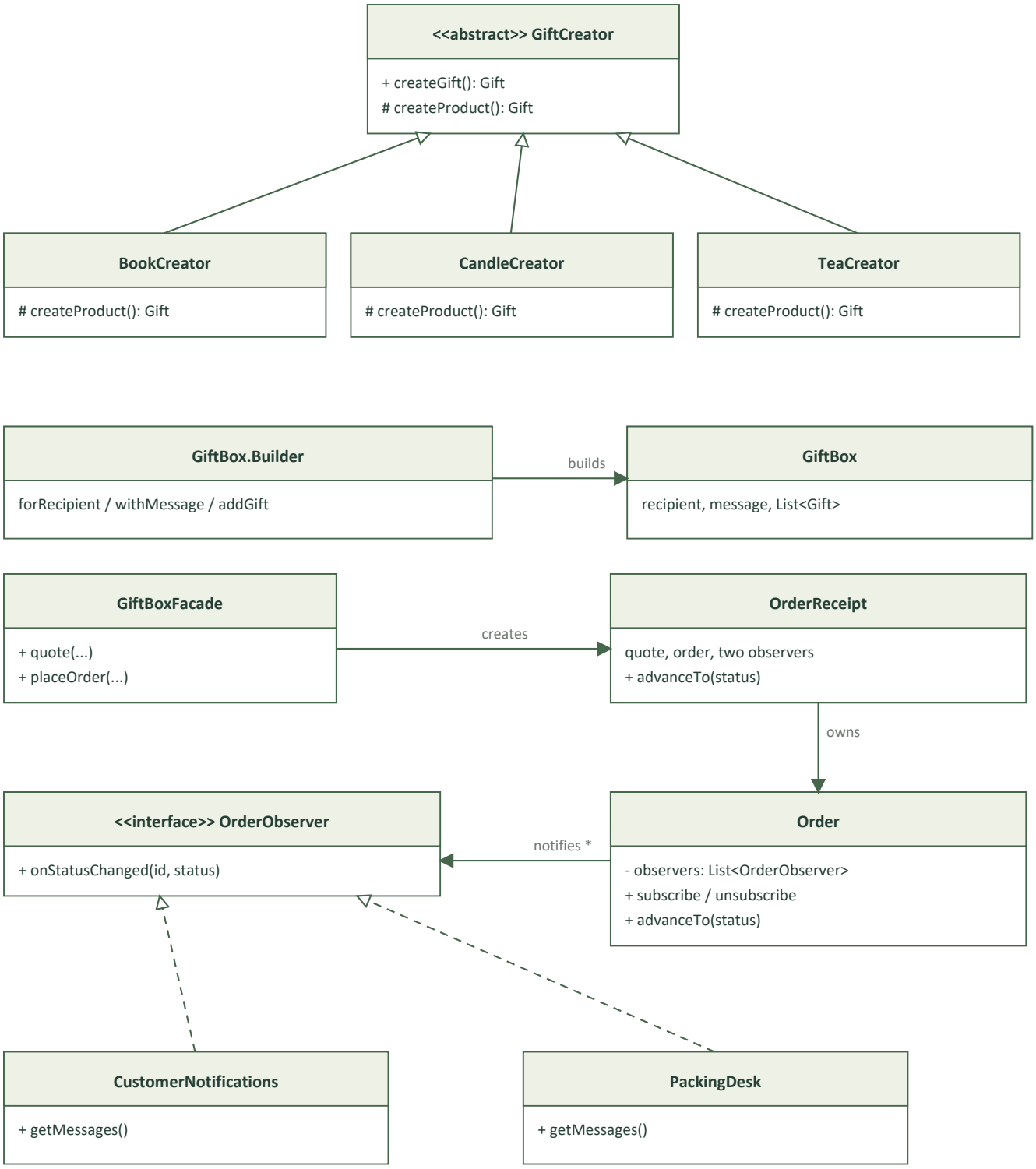


How to read the relationships

GiftDecorator is both a Gift and a holder of another Gift. This is the central structural relationship of Decorator. PriceCalculator holds a DiscountStrategy; concrete policies satisfy the same operation. Replacing that reference changes behavior without changing the gifts or the summation algorithm.

Editable sources: docs/uml/classes.puml. Separate vector versions: decorator.svg and strategy.svg. Actual source: api, decorator, pricing and strategy packages.

UML: creation and order coordination

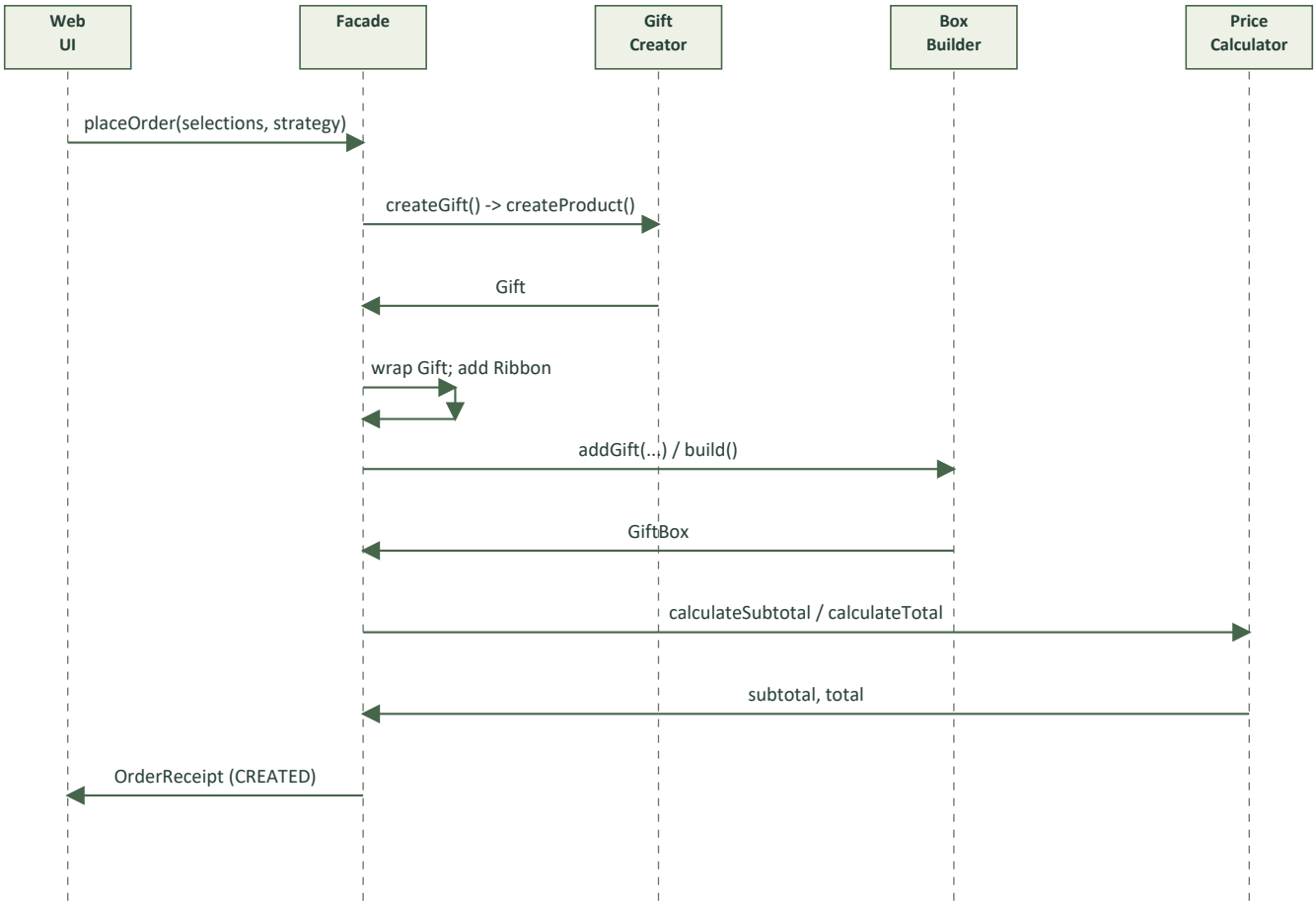


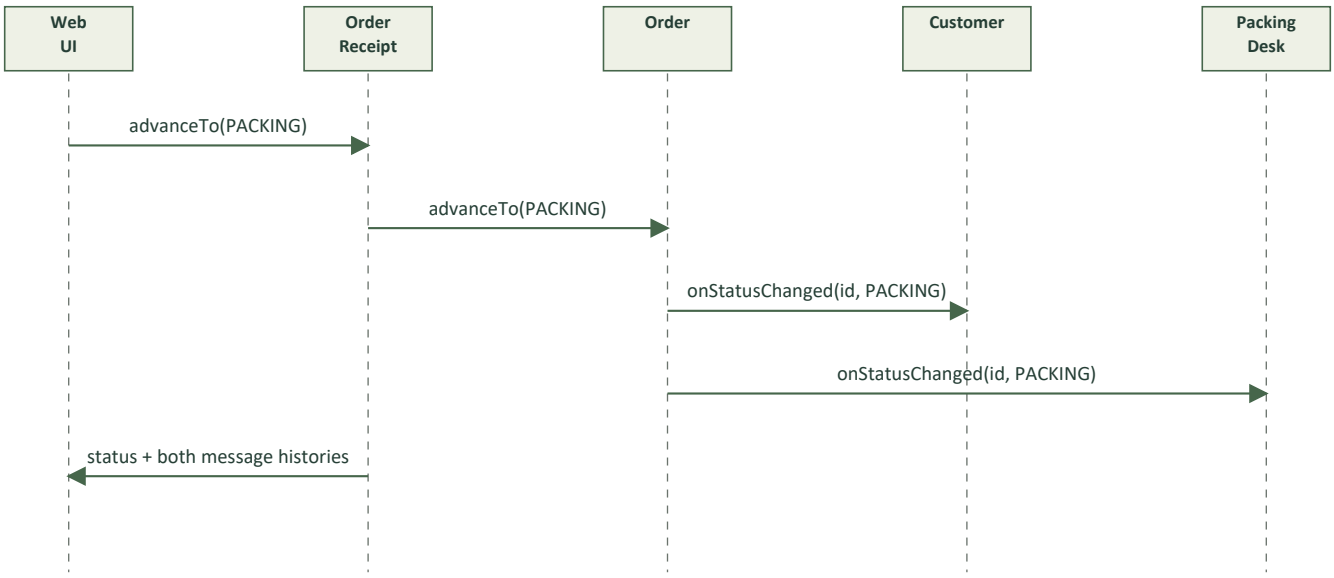
Factory Method: concrete creators override createProduct. Builder: a separate temporary builder produces a GiftBox. Facade: GiftBoxFacade uses these creation types, decorators and PriceCalculator (shown on the previous page) and creates an OrderReceipt.

Observer: OrderReceipt attaches CustomerNotifications and PackingDesk to its Order. Order knows only the observer interface, so the publisher does not contain separate calls for each concrete subscriber. All represented relationships correspond to the delivered Java source.

UML: request and event sequences

The Web/UI lifeline combines the browser request and HTTP adapter to keep the domain call sequence readable. createGift calls the overridden factory method; quote follows the same assembly path without constructing an order.





PACKING is shown. The READY transition repeats the same callback sequence. Customer and packing history remain separate. A request that skips a state is rejected before an event is published. Editable source: docs/uml/sequence.puml.

Application screenshots

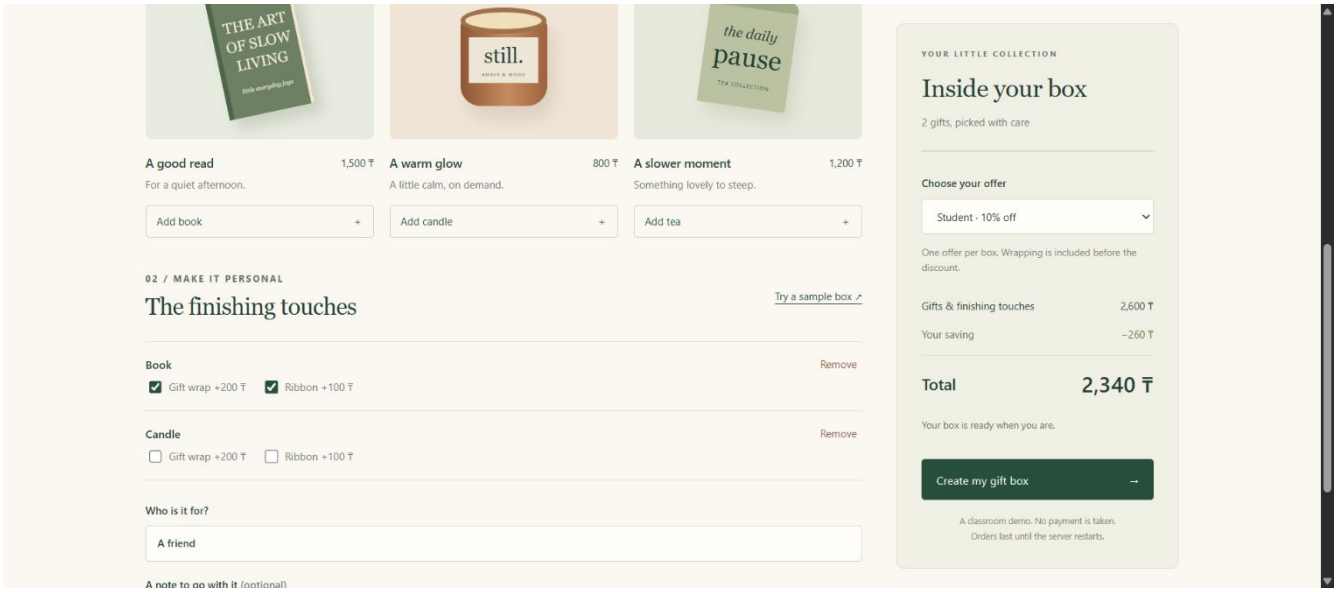


Figure 1. Actual browser run: sample box, both book decorations selected, Student offer. Java returned subtotal 2600 KZT and total 2340 KZT. Source screenshot: 01-builder.jpg.

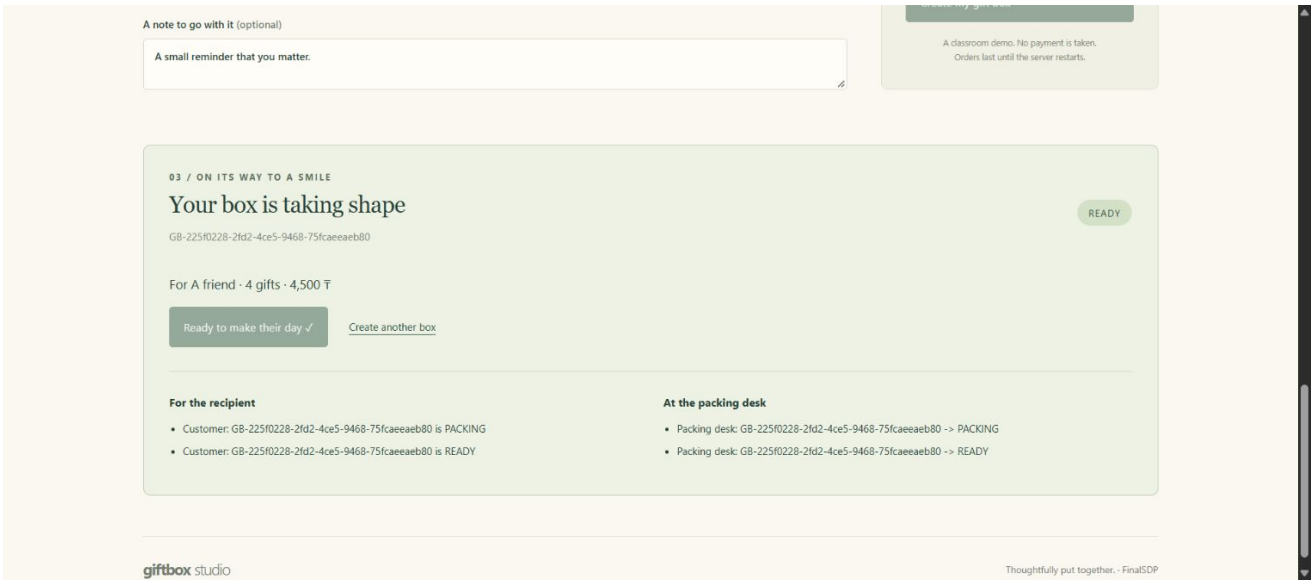


Figure 2. Actual order after CREATED → PACKING → READY. The threshold example has four gifts, subtotal 5000 KZT and total 4500 KZT. Both subscriber histories contain PACKING and READY. Source screenshot: 03-order-ready.jpg.

The screenshots are captured from the running application, not mockups. Additional threshold and mobile screenshots are included in docs/screenshots. At a 390 px viewport (375 px content area with the scrollbar), the observed page had no horizontal overflow.

Verification and reproducibility

Check	Result	Evidence
User part	12 passed / 0 failed	test/giftbox/PartTests.java
Full project	12 passed / 0 failed	test/giftbox/ProjectTests.java
HTTP integration	20 passed / 0 failed	test/http_smoke.py
Browser workflow	Verified interactively	Sample pricing, threshold, order creation, PACKING, READY, two histories
Responsive layout	390 px viewport checked	Content width matched viewport content width; mobile screenshot
JavaScript syntax	Passed	Node --check web/app.js

Important test cases

The user-part suite covers wrapper composition, unchanged base objects, independent Gift implementations, runtime strategy replacement, once-per-box discounting, rounding, empty lists, nulls, negative amounts, invalid strategy output and overflow. The project suite covers creator extensibility, builder snapshots, text/item limits, threshold boundaries, unsubscribe, invalid state changes and full six-pattern integration.

HTTP checks exercise the real server: valid quotes, JSON escaping, created orders, both notification streams, invalid transitions, unknown gifts and offers, empty input, excessive item count, missing orders, method restrictions, oversized bodies and cross-origin rejection. These checks do not establish production security or multi-user scalability.

Run from the source archive

```
# Windows PowerShell, from the extracted project folder
powershell -ExecutionPolicy Bypass -File ./build.ps1 -Test -Run
powershell -ExecutionPolicy Bypass -File ./build.ps1 -Web
# Open http://127.0.0.1:8080
# In a second terminal, with Python 3:
python test/http_smoke.py
```

Verification environment: Windows, JDK 25 compiling with --release 11, Chrome. JDK 11 itself and Linux/macOS launch scripts were not executed in this environment. Python is required only for the optional HTTP tests, not for running the application.

Conclusion and further work

Conclusion

GiftBox Studio demonstrates six distinct patterns in one working application. Factory Method creates products; Decorator enriches individual gifts; Builder produces a box; Strategy calculates the selected offer; Facade connects the use case; Observer reacts to order progress. The browser interface makes their combined effect visible, while Java tests make important boundary behavior reproducible.

The strongest part of the design is the separation of gift structure, pricing and order events. The threshold offer demonstrates a behavior change that is more than a different percentage. The project remains intentionally small enough to trace a browser action to a specific method during the defense.

Further work

Improvement	Reason and affected area
Persistent orders	Add a repository and storage so restarting does not lose orders. Keep persistence outside pricing and decoration.
Reliable notifications	Isolate observer failures and, if needed, enqueue external delivery. Add failure tests before sending real email.
Richer workflow	Replace enum-order validation with explicit allowed transitions if cancellation or delivery branches are introduced.
Production deployment	Add identity, authorization, concurrency controls, durable storage and operational monitoring before exposing the server publicly.
Catalogue extension	Implement another GiftCreator and add its UI card. Keep the Gift interface stable and test decoration compatibility.

Team responsibility and defense

Preparation allocation: Mukhametov Temirlan - Strategy and Decorator; Saparali Shalkar - Factory Method and Builder; Nurbayev Murager - Observer and Facade. The latter two assignments can be exchanged by the team. This describes responsibility for study and further work, not a verified history of independent authorship.

The delivered implementation was prepared with AI assistance. The assignment separately requires each student to implement at least two patterns individually. This report and a complete repository do not certify that requirement. Each member must understand, develop and defend their own contribution in accordance with course rules.

References and submission sources

1. Final Project Defense Requirements.pdf, provided course handout, page 1 (requirements).
2. Instructor upload message: source ZIP/RAR, report with GitHub link and at least six pages, additional UML/schemas.
3. Delivered Java sources, CONTRACT.md, TEAM.md and test suites. Repository link appears on page 1.